UNIVERSIDADE FEDERAL DE ITAJUBÁ

SISTEMAS DE INFORMAÇÃO

RELATORIO DO MIDDLEWARE: APACHE PULSAR

2021017110 - João Leão Guedes Filho

XRSC09 - SISTEMAS DISTRIBUÍDOS

Prof. Rafael Frinhani

INSTITUTO DE MATEMÁTICA E COMPUTAÇÃO
UNIFEI - Itajubá

2026

UNIVERSIDADE FEDERAL DE ITAJUBÁ
INSTITUTO DE MATEMÁTICA E COMPUTAÇÃO
SISTEMAS DE INFORMAÇÃO
XRSC09 - SISTEMAS DISTRIBUÍDOS

Relatorio do Middleware: Apache Pulsar

1 Referencial Teorico

1.1 Paradigma de comunicacao

O Apache Pulsar adota o paradigma de *Mensageria e Streaming*, com padrao de interacao *Publish-Subscribe (Pub/Sub)*. Nesse modelo, produtores publicam em topicos e consumidores recebem mensagens por assinaturas, sem dependencia direta entre os processos.

Esse mecanismo promove desacoplamento:

- **no tempo**: emissor e receptor nao precisam estar ativos simultaneamente;
- **no espaco**: os componentes nao precisam conhecer localizacao/identidade um do outro.

1.2 Problema que o middleware resolve

Em sistemas distribuidos, comunicacao direta entre servicos aumenta acoplamento e dificulta evolucao do sistema. O middleware atua como camada de intermediacao para:

- roteamento de mensagens;
- persistencia e entrega confiavel;
- escalabilidade de produtores e consumidores.

1.3 Historico e contexto de uso

O Pulsar foi desenvolvido inicialmente no Yahoo e posteriormente doado a Apache Software Foundation. A plataforma e usada em cenarios com alto volume de eventos e baixa latencia, como integracao de microsservicos, processamento de eventos em tempo real, IoT e plataformas financeiras.

1.4 Exemplos de aplicacao

No contexto estudado, destacam-se cenarios de:

- divulgacao automatica de oportunidades de investimento em tempo real;
- integracao de aplicacoes empresariais com sistemas legados;
- monitoramento de dispositivos IoT e processamento de telemetria.

1.5 Arquitetura da tecnologia

No nivel conceitual, a arquitetura do Pulsar separa computacao e armazenamento:

- **Serving Layer**: brokers stateless para conexao, roteamento e entrega;
- **Storage Layer**: Apache BookKeeper para persistencia em ledgers;
- **Metadata Store**: coordenacao e metadados.

Essa separacao favorece escalabilidade independente entre processamento e armazenamento, caracteristica importante para sistemas distribuidos modernos.

2 Referencial Teorico

2.1 Paradigma de comunicacao

O Apache Pulsar adota o paradigma de *Mensageria e Streaming*, com padrao de interacao *Publish-Subscribe (Pub/Sub)*. Nesse modelo, produtores publicam em topicos e consumidores recebem mensagens por assinaturas, sem dependencia direta entre os processos.

Esse mecanismo promove desacoplamento:

- **no tempo**: emissor e receptor nao precisam estar ativos simultaneamente;
- **no espaco**: os componentes nao precisam conhecer localizacao/identidade um do outro.

2.2 Problema que o middleware resolve

Em sistemas distribuidos, comunicacao direta entre servicos aumenta acoplamento e dificulta evolucao do sistema. O middleware atua como camada de intermediacao para:

- roteamento de mensagens;
- persistencia e entrega confiavel;
- escalabilidade de produtores e consumidores.

2.3 Historico e contexto de uso

O Pulsar foi desenvolvido inicialmente no Yahoo e posteriormente doado a Apache Software Foundation. A plataforma e usada em cenarios com alto volume de eventos e baixa latencia, como integracao de microsservicos, processamento de eventos em tempo real, IoT e plataformas financeiras Apache Pulsar (2026b).

© UNIFEI-IMC - 2026 - 1º Semestre

2.4 Exemplos de aplicacao

No contexto estudado, destacam-se cenarios de:

- divulgacao automatica de oportunidades de investimento em tempo real;
- integracao de aplicacoes empresariais com sistemas legados;
- monitoramento de dispositivos IoT e processamento de telemetria.

2.5 Arquitetura da tecnologia

No nivel conceitual, a arquitetura do Pulsar separa computacao e armazenamento:

- **Serving Layer**: brokers stateless para conexao, roteamento e entrega;
- **Storage Layer**: Apache BookKeeper para persistencia em ledgers;
- **Metadata Store**: coordenacao e metadados.

Essa separacao favorece escalabilidade independente entre processamento e armazenamento, caracteristica importante para sistemas distribuidos modernos Apache Pulsar (2026a).

3 Desenvolvimento

3.1 Modelagem da solucao

O prototipo foi implementado em Python, com orquestracao via Docker Compose, e segue arquitetura cliente-servidor com middleware de mensageria.

Elemento	Papel no sistema
`pulsar`	Broker de mensagens responsavel por rotear requests e responses.
`server`	Processo que consome requisicoes, executa operacoes e publica respostas.
`client`	Processo que envia requisicoes e aguarda resposta por topico de retorno.
`PulsarRpcClient`	Classe cliente para fluxo request/reply.
`Request` e `Response`	Contratos de mensagem serializados em JSON.

Tabela 1: Servicos e componentes principais do prototipo.

Item	Identificacao
Topico de request (fixo)	`prototype-requests`
Topico de response (dinamico)	`replies-{uuid}`
Subscription do servidor	`prototype-server` (Shared)
Subscription do cliente	`client-{uuid}`

Tabela 2: Topicos e assinaturas do PrototipoSeminario.

3.2 Fluxo de comunicacao

1. O cliente cria producer para requests e subscription de resposta.
2. O cliente envia `Request` com `request_id`, operacao, payload e `reply_to`.
3. O servidor consome a mensagem no topico de request.
4. O servidor executa `echo`, `write_file` ou `calculate`.
5. O servidor publica `Response` no topico de retorno informado.
6. O cliente consome e valida a resposta pelo `request_id`.

3.3 Bibliotecas, parametros e comandos

Bibliotecas e ferramentas: Python 3.10, `pulsar-client`, Docker e Docker Compose PyPI (2026).

Parametros principais:

- `PULSAR_URL`: endereco do broker;
- `REQUEST_TOPIC`: topico de requisicoes;
- `SERVER_FILE_DIR`: diretorio de escrita no servidor.

3.4 Diagramas

Os diagramas foram exportados para PNG e inseridos diretamente no relatorio.

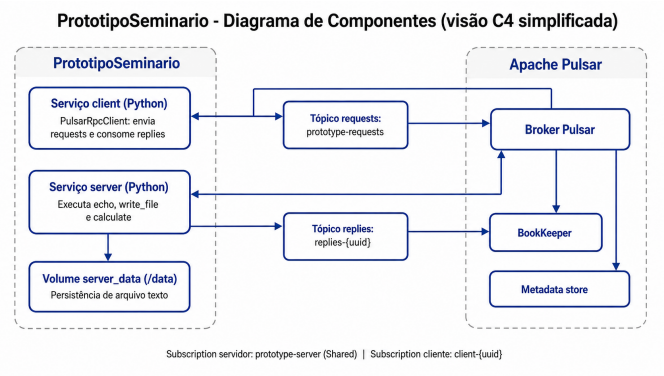

Figura 1: Diagrama de componentes do PrototipoSeminario.

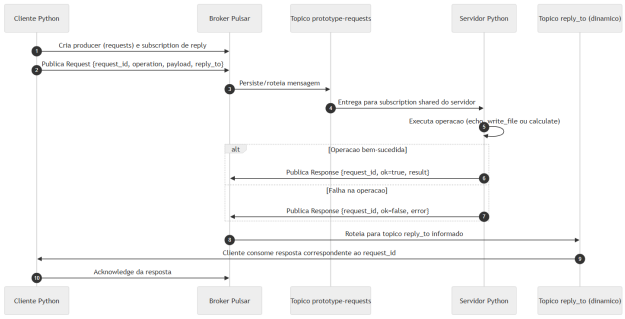

Figura 2: Diagrama de sequencia do fluxo request/reply.

4 Resultados e Avaliacao

4.1 Validacao funcional do prototipo

Foram executados testes fim-a-fim com Apache Pulsar em funcionamento, validando as tres operacoes obrigatorias e um cenario de erro controlado.

ID	Objetivo do teste	Status
T1	Validar resposta de texto (`echo`)	PASS
T2	Validar alteracao de arquivo (`write_file`)	PASS
T3	Validar calculo de funcao (`calculate`)	PASS
T4	Validar tratamento de erro para funcao invalida	PASS

Tabela 3: Resumo dos testes executados.

4.2 Evidencias de execucao

A Tabela ?? apresenta os principais registros coletados durante a execucao do prototipo.

Evidencia	Resultado observado
Retorno do cliente (teste `echo`)	`ok=true` e mensagem recebida do servidor confirmando a conectividade.
Retorno do cliente (teste `calculate`)	`ok=true`, funcao `multiply` e resultado numerico 42.
Log do servidor	Registro de requisicoes com `request_id` e `reply_to` para `echo`, `write_file` e `calculate`.
Persistencia no servidor	Arquivo `registro_e2e.txt` criado e atualizado no diretorio de dados do servidor.

Tabela 4: Evidencias tecnicas de execucao.

4.3 Avaliacao critica

- **Vantagens:** desacoplamento entre processos, escalabilidade horizontal e persistencia nativa de mensagens.
- **Limitacoes:** maior complexidade operacional quando comparado a comunicacao direta, alem da necessidade de monitoramento do broker em ambiente real.
- **Cenarios adequados:** integracao assincrona entre servicos, processamento de eventos e distribuicao de mensagens para multiplos consumidores.
- **Cenarios menos indicados:** sistemas pequenos com baixa concorrencia e comunicacao predominantemente sincrona.

4.4 Conclusao

O prototipo atendeu aos requisitos obrigatorios da disciplina para comunicacao distribuida com middleware, com evidencias tecnicas de funcionamento e resultados consistentes nos testes executados.

Referências

Apache Pulsar (2026a). Python client library. https://pulsar.apache.org/docs/client-libraries/python/. Acesso em: 04 maio 2026.

Apache Pulsar (2026b). Run pulsar in standalone mode with docker. https://pulsar.apache.org/docs/en/standalone-docker/. Acesso em: 04 maio 2026.

PyPI (2026). pulsar-client. https://pypi.org/project/pulsar-client/. Acesso em: 04 maio 2026.